Connectly — Solution Architecture

Hyper-Local Community Job Marketplace | CPUT Capstone Architecture Specification

Belhar, Cape Town | Tiered Client-Server | Target: ~R745/mo

Architecture Overview: A scalable, resource-optimized multi-tier solution bridging job seekers and household employers in Belhar. Built for low data consumption on Android devices using client caching, REST APIs, JSON payloads, normalized MySQL database, and secure POPIA-compliant authentication.

1. CLIENTS & UI LAYER

Job Seekers (Belhar)

Low-bandwidth mobile & web views designed for resource-constrained Android devices.

Household Employers

Web application interface for posting gig listings, evaluating candidates, and payments.

System Admin Dashboard

Portal for user identity verification, dispute resolution, and platform usage analytics.

Presentation Tech

HTML5 | CSS3 | JavaScript | Client Caching

2. APPLICATION & SERVICES LAYER

API Gateway & Router

HTTPS enforcement, JWT bearer token verification, rate limiting, and RBAC security routing.

Core Service Modules

- **Auth & Identity:** Registration, JWT generation, POPIA consent tracking.
- **Geo-Matching Engine:** Radius matching for Belhar job seekers & local postings.
- **Job Listing Service:** Gig creation, category filtering, lifecycle management.
- **Messaging & Alerts:** Instant transactional push & SMS job updates.
- **Payment & Escrow:** Secure gateway routing & transaction confirmation.

3. DATA & INTEGRATION

Primary Database

Normalized MySQL 8.0 relational database storing users, jobs, ratings, and logs.

Session & Cache Store

Redis Cache for session tokens, rapid listing queries, and transient state.

External API Integrations

- **Payments:** PayFast / Ozow / SnapScan
- **SMS / Alerts:** Twilio / Africa's Talking
- **Geolocation:** Google Maps / OpenStreetMap

4. INFRASTRUCTURE & DEVOPS

Cloud Deployment

Containerized hosting via **Docker** deployed on Render / AWS Engine.

CI/CD Pipeline

GitHub Actions automated builds, integration tests, and production deployments.

Security & Compliance

- TLS 1.3 Transport Encryption
- Parameterized SQL Statements
- POPIA Data Handling Compliance

DATA FLOW ARCHITECTURE

1. Client issues REST API request over HTTPS via JSON payloads.
2. API Gateway validates JWT authentication and routes request.
3. Microservices query Redis cache / MySQL DB and execute business logic.
4. Notifications & payment transactions trigger third-party API webhooks.

SECURITY & POPIA COMPLIANCE

- End-to-end SSL/TLS encryption for all data in transit.
- Passwords hashed with salted bcrypt algorithms.
- Strict data privacy measures meeting South African POPIA regulations.
- Role-Based Access Control (RBAC) separating seekers, employers, and admins.

COST & PERFORMANCE EFFICIENCY

- Engineered for a lean monthly operating budget (~R745/month).
- Client-side UI caching reduces unnecessary network requests.
- Lightweight JSON payloads optimize performance on 3G/4G mobile networks.